

THE MAHARAJA SAYAJIRAO UNIVERSITY OF BARODA

FACULTY OF TECHNOLOGY AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LABORATORY WORK SUBMISSION

ARTIFICIAL INTELLIGENCE (AI)

Practical 1, 2, 3: Search Algorithms & Hill Climbing Variants

Academic Year: 2026 – 2027

Student Name:	KHATRI OM KUMAR
PRN:	8024056707
Roll No:	36
Class:	BE-III (GIA)
Branch:	Computer Science & Engineering
Subject:	Artificial Intelligence (AI)

Practical 1, 2, 3: Implement Search Algorithms (3 weeks)

Path Finding on a City Map

1. Check the cities.py file. That is the map of the cities

→ graph = {

```
"Ahmedabad": [("Vadodara", 110), ("Rajkot", 215), ("Udaipur", 260), ("Indore", 400)],
"Vadodara": [("Ahmedabad", 110), ("Surat", 155), ("Indore", 390), ("Udaipur", 330)],
"Surat": [("Vadodara", 155), ("Mumbai", 285), ("Nashik", 260)],
"Mumbai": [("Surat", 285), ("Nashik", 170), ("Pune", 150)],
"Pune": [("Mumbai", 150), ("Nashik", 210), ("Solapur", 250), ("Hyderabad", 560)],

"Nashik": [("Mumbai", 170), ("Surat", 260), ("Pune", 210), ("Aurangabad", 180), ("Indore", 420
)],
"Aurangabad": [("Nashik", 180), ("Nagpur", 480), ("Hyderabad", 560), ("Indore", 410)],
"Rajkot": [("Ahmedabad", 215), ("Bhuj", 230)],
"Bhuj": [("Rajkot", 230), ("Ahmedabad", 335)],

"Udaipur": [("Ahmedabad", 260), ("Vadodara", 330), ("Jaipur", 395), ("Indore", 385), ("Kota", 2
85)],
"Jaipur": [("Udaipur", 395), ("Delhi", 280), ("Agra", 240), ("Kota", 250)],
"Delhi": [("Jaipur", 280), ("Agra", 233), ("Chandigarh", 245), ("Lucknow", 555)],
"Agra": [("Delhi", 233), ("Jaipur", 240), ("Gwalior", 120), ("Kanpur", 285)],
"Gwalior": [("Agra", 120), ("Jhansi", 100), ("Bhopal", 420)],
"Jhansi": [("Gwalior", 100), ("Bhopal", 330), ("Kanpur", 220)],
"Kanpur": [("Agra", 285), ("Jhansi", 220), ("Lucknow", 90), ("Prayagraj", 200)],
"Lucknow": [("Delhi", 555), ("Kanpur", 90), ("Prayagraj", 200), ("Varanasi", 315)],
"Prayagraj": [("Kanpur", 200), ("Lucknow", 200), ("Varanasi", 125)],
"Varanasi": [("Prayagraj", 125), ("Lucknow", 315), ("Patna", 255)],
"Patna": [("Varanasi", 255), ("Ranchi", 330), ("Kolkata", 585)],
"Ranchi": [("Patna", 330), ("Kolkata", 400), ("Raipur", 560)],
"Kolkata": [("Patna", 585), ("Ranchi", 400), ("Bhubaneswar", 440)],
"Bhubaneswar": [("Kolkata", 440), ("Visakhapatnam", 440), ("Raipur", 530)],
```

```

"Visakhapatnam": [("Bhubaneswar", 440), ("Vijayawada", 350), ("Hyderabad", 620)],
"Vijayawada": [("Visakhapatnam", 350), ("Hyderabad", 275), ("Chennai", 450)],

"Hyderabad": [("Pune", 560), ("Aurangabad", 560), ("Nagpur", 500), ("Vijayawada", 275), ("Bengaluru", 570), ("Chennai", 630), ("Visakhapatnam", 620)],

"Nagpur": [("Indore", 445), ("Bhopal", 350), ("Aurangabad", 480), ("Hyderabad", 500), ("Raipur", 285)],

"Indore": [("Ahmedabad", 400), ("Vadodara", 390), ("Udaipur", 385), ("Bhopal", 195), ("Nagpur", 445), ("Nashik", 420), ("Aurangabad", 410)],
"Bhopal": [("Indore", 195), ("Nagpur", 350), ("Jhansi", 330), ("Gwalior", 420)],
"Raipur": [("Nagpur", 285), ("Ranchi", 560), ("Bhubaneswar", 530)],
"Chandigarh": [("Delhi", 245)],
"Kota": [("Jaipur", 250), ("Udaipur", 285)],
"Bengaluru": [("Hyderabad", 570), ("Chennai", 345)],
"Chennai": [("Bengaluru", 345), ("Vijayawada", 450), ("Hyderabad", 630)],
"Solapur": [("Pune", 250)]
}

```

→ Print the path, total cost.

→ Track and print the number of nodes explored for each algorithm during the entire search process (this gives you an estimate of the time complexity) and also track and print the maximum number of nodes stored in the Queue/Stack at any point in time (this gives you an estimate of the space complexity).

2. Implement BFS

```

from collections import deque
from city_map import road_network

```

```

def calculate_cost(route):
    total_dist = 0
    for k in range(len(route) - 1):
        for adj_city, dist in road_network[route[k]]:
            if adj_city == route[k + 1]:
                total_dist += dist
                break
    return total_dist

def bfs(source, destination):
    frontier = deque([[source]])
    explored = set()
    nodes_explored = 0
    max_stored = len(frontier)

    while frontier:
        route = frontier.popleft()
        current_node = route[-1]

        if current_node == destination:
            print("Path :", " -> ".join(route))
            print("Cost :", calculate_cost(route))
            print("Nodes Explored (Time Complexity) :", nodes_explored)
            print("Max Nodes Stored (Space Complexity) :", max_stored)
            return

        if current_node not in explored:
            explored.add(current_node)
            nodes_explored += 1

            for adj_city, dist in road_network[current_node]:
                extended_route = route + [adj_city]
                frontier.append(extended_route)

            if len(frontier) > max_stored:
                max_stored = len(frontier)

    source = input("Start City : ")
    destination = input("Goal City : ")

    bfs(source, destination)

```

Output :

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> python .\breadth_first.py
Start City : Ahmedabad
Goal City : Chennai
Path : Ahmedabad -> Indore -> Nagpur -> Hyderabad -> Chennai
Cost : 1975
Nodes Explored (Time Complexity) : 26
Max Nodes Stored (Space Complexity) : 37
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> █
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> python .\breadth_first.py
Start City : Mumbai
Goal City : Delhi
Path : Mumbai -> Surat -> Vadodara -> Udaipur -> Jaipur -> Delhi
Cost : 1445
Nodes Explored (Time Complexity) : 25
Max Nodes Stored (Space Complexity) : 31
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> █
```

3. Implement DFS

```
from collections import deque
from city_map import road_network

def calculate_cost(route):
    total_dist = 0
    for k in range(len(route) - 1):
        for adj_city, dist in road_network[route[k]]:
            if adj_city == route[k + 1]:
                total_dist += dist
                break
    return total_dist
```

```

def dfs(source, destination):
    frontier = deque([[source]])
    explored = set()
    nodes_explored = 0
    max_stored = len(frontier)

    while frontier:
        route = frontier.pop() # LIFO stack behavior for Depth-First Search
        current_node = route[-1]

        if current_node == destination:
            print("Path :", " -> ".join(route))
            print("Cost :", calculate_cost(route))
            print("Nodes Explored (Time Complexity) :", nodes_explored)
            print("Max Nodes Stored (Space Complexity) :", max_stored)
            return

        if current_node not in explored:
            explored.add(current_node)
            nodes_explored += 1

            for adj_city, dist in road_network[current_node]:
                extended_route = route + [adj_city]
                frontier.append(extended_route)

            if len(frontier) > max_stored:
                max_stored = len(frontier)

    source = input("Start City : ")
    destination = input("Goal City : ")

    dfs(source, destination)

```

4.

Output :

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> python .\depth_first.py
Start City : Ahmedabad
Goal City : Chennai
Path : Ahmedabad -> Indore -> Aurangabad -> Hyderabad -> Visakhapatnam -> Vijayawada -> Chennai
Cost : 2790
Nodes Explored (Time Complexity) : 6
Max Nodes Stored (Space Complexity) : 21
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION>

```

```
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> python .\depth_first.py
Start City : Mumbai
Goal City : Delhi
Path : Mumbai -> Pune -> Hyderabad -> Visakhapatnam -> Bhubaneswar -> Raipur -> Ranchi -> Kolkata -> Patna
-> Varanasi -> Lucknow -> Prayagraj -> Kanpur -> Jhansi -> Bhopal -> Gwalior -> Agra -> Jaipur -> Delhi
Cost : 6425
Nodes Explored (Time Complexity) : 32
Max Nodes Stored (Space Complexity) : 43
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> █
```

5. A* Search

```
import heapq
from cities import graph

# Straight-line distance / heuristic values
heuristics = {}

def a_star(start, goal):
    # Queue stores tuples of (f_cost, g_cost, path)
    # f_cost = g_cost + h_cost
    queue = [(0, 0, [start])]
    visited = []
    nodes_explored = 0
    max_stored = len(queue)

    while queue:
        f_cost, g_cost, path = heapq.heappop(queue)
        city = path[-1]

        if city == goal:
            print("Path :", " -> ".join(path))
            print("Cost :", find_cost(path))
            print("Nodes Explored (Time Complexity) :", nodes_explored)
            print("Max Nodes Stored (Space Complexity) :", max_stored)
            return

        if city not in visited:
            visited.append(city)
            nodes_explored += 1

            if city in graph:
                for neighbour, cost in graph[city]:
                    new_path = list(path)
                    new_path.append(neighbour)
                    new_g_cost = g_cost + cost
                    h_cost = heuristics.get(neighbour, 0)
                    new_f_cost = new_g_cost + h_cost
                    heapq.heappush(queue, (new_f_cost, new_g_cost, new_path))

        if len(queue) > max_stored:
            max_stored = len(queue)
```

```

def find_cost(path):
    total = 0
    for i in range(len(path) - 1):
        if path[i] in graph:
            for neighbour, cost in graph[path[i]]:
                if neighbour == path[i + 1]:
                    total += cost
    return total

start = input("Start City : ")
goal = input("Goal City : ")

a_star(start, goal)

```

Output :

Problems Output Debug Console **Terminal** Ports

```

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> python .\astar_search.py
Start City : Ahmedabad
Goal City : Chennai
Path : Ahmedabad -> Vadodara -> Surat -> Mumbai -> Pune -> Hyderabad -> Chennai
Cost : 1890
Nodes Explored (Time Complexity) : 33
Max Nodes Stored (Space Complexity) : 30
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION>

```

Problems Output Debug Console **Terminal** Ports

```

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> python .\astar_search.py
Start City : Mumbai
Goal City : Delhi
Path : Mumbai -> Surat -> Vadodara -> Udaipur -> Jaipur -> Delhi
Cost : 1445
Nodes Explored (Time Complexity) : 27
Max Nodes Stored (Space Complexity) : 32
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION>

```

➤ Hill Climbing Variants on the 8-Queens Problem

Problem Statement :

Represent a chessboard using an array of length 8 where the index is the column and the value is the row of the queen. Example: [4,6,1,5,2,0,3,7]. One queen must exist in every column. The heuristic $h(n)$ is the number of attacking pairs of queens. The goal state has $h(n)=0$.

File	Contents	Task
queens_common.py	heuristic(), get_neighbours(), random_state(), print_board(), print_result()	Tasks 1 and 2
steepest_ascent.py	Steepest Ascent Hill Climbing	Task 3
first_choice.py	First-Choice Hill Climbing	Task 4
stochastic.py	Stochastic Hill Climbing	Task 5
random_restart.py	Random Restart Hill Climbing	Task 6
run_experiments.py	20 runs of each algorithm and the observation table	Experiments

Each program calls `random.seed(...)` before it generates its starting board, so every run produces exactly the same output. All results shown in this report are therefore reproducible.

Source code

1. queens_common.py — Tasks 1 and 2

Helper functions shared by all four programs. This file is imported, never run on its own.

```
# Common helper functions used by all four hill climbing programs.
# HOW WE STORE THE BOARD: The board is a simple list of 8 numbers.
# index = column number, value = row number of the queen in that column.
# Example: [4, 6, 1, 5, 2, 0, 3, 7]
```

```

# column 0 has a queen in row 4, column 1 has a queen in row 6 ... and so on.
# Because a list has only one value per index, every column always has exactly one queen.
# That is why we only need to check rows and diagonals.

import random

BOARD_SIZE = 8

# ---- conflict counter (heuristic) ----
def count_conflicts(board):
    # Return the number of pairs of queens that attack each other
    attacks = 0
    for i in range(BOARD_SIZE):
        for j in range(i + 1, BOARD_SIZE):
            same_row = board[i] == board[j]
            same_diag = abs(board[i] - board[j]) == (j - i)
            if same_row or same_diag:
                attacks += 1
    return attacks

# ---- neighbor generator ----
def generate_neighbors(board):
    # Produce every board reachable by moving a single queen to another row within its own column
    result = []
    for col in range(BOARD_SIZE):
        for row in range(BOARD_SIZE):
            if row != board[col]:
                modified = board[:]
                modified[col] = row
                result.append(modified)
    return result

# ---- random board creator ----
def create_random_board():
    # Generate a board with each queen placed on a random row
    return [random.randint(0, BOARD_SIZE - 1) for _ in range(BOARD_SIZE)]

# ---- display helpers ----
def display_board(board):

```

```

# Render the board to stdout. Q = queen, . = empty
for r in range(BOARD_SIZE):
    row_str = ""
    for c in range(BOARD_SIZE):
        row_str += " Q" if board[c] == r else " ."
    print(row_str)

def show_summary(algo_name, initial, final, h_val, iters, evals):
    # Print a standardised result block used by every algorithm
    print()
    print("==== " + algo_name + " =====")
    print("Start board :", initial, " h =", count_conflicts(initial))
    print("Final board :", final, " h =", h_val)
    if h_val == 0:
        print("Result      : SUCCESS")
    else:
        print("Result      : FAILURE (stuck at a local optimum)")
    print("Iterations  :", iters)
    print("Neighbour evaluations :", evals)
    print()
    display_board(final)

```

2. steepest_ascent.py — Task 3

```

# TASK 3 : Steepest Ascent Hill Climbing
# Idea: 1. Look at ALL 56 neighbours. 2. Move to single best one (lowest h). 3. Stop if no improvement.
import random
from nqueens_utils import count_conflicts, generate_neighbors, create_random_board, show_summary

def steepest_climb(board, verbose=False):
    iterations = 0
    evaluations = 0
    current_h = count_conflicts(board)
    while current_h > 0:
        optimal = board
        optimal_h = current_h
        for neighbor in generate_neighbors(board):
            neighbor_h = count_conflicts(neighbor)
            evaluations += 1

```

```

        if neighbor_h < optimal_h:
            optimal = neighbor
            optimal_h = neighbor_h
    # no improvement means we hit a plateau or local minimum
    if optimal_h >= current_h:
        break
    board = optimal
    current_h = optimal_h
    iterations += 1
    if verbose:
        print("step", iterations, ":", board, " h =", current_h)
    return board, current_h, iterations, evaluations

if __name__ == "__main__":
    random.seed(23)
    initial = create_random_board()
    print("Steepest Ascent Hill Climbing")
    print("Start board :", initial, " h =", count_conflicts(initial))
    final, h_val, iters, evals = steepest_climb(initial, verbose=True)
    show_summary("Steepest Ascent Hill Climbing", initial, final, h_val, iters, evals)

```

Output :

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> python .\steepest_hill.py
Steepest Ascent Hill Climbing
Start board : [4, 1, 0, 4, 6, 6, 5, 2]  h = 5
step 1 : [4, 1, 0, 4, 6, 1, 5, 2]  h = 3
step 2 : [4, 7, 0, 4, 6, 1, 5, 2]  h = 1
step 3 : [3, 7, 0, 4, 6, 1, 5, 2]  h = 0

==== Steepest Ascent Hill Climbing ====
Start board : [4, 1, 0, 4, 6, 6, 5, 2]  h = 5
Final board : [3, 7, 0, 4, 6, 1, 5, 2]  h = 0
Result      : SUCCESS
Iterations  : 3
Neighbour evaluations : 168

. . Q . . . .
. . . . . Q .
. . . . . . Q
Q . . . . .
. . . Q . . .
. . . . . Q .
. . . . Q . .
. Q . . . . .

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION>

```

3. first_choice.py — Task 4

```
# TASK 4 : First-Choice Hill Climbing
# Idea: 1. Shuffle neighbours in random order. 2. Move to FIRST improving neighbour. 3. Stop if none.
import random
from nqueens_utils import count_conflicts, generate_neighbors, create_random_board, show_summary

def first_choice_climb(board, verbose=False):
    iterations = 0
    evaluations = 0
    current_h = count_conflicts(board)
    while current_h > 0:
        candidates = generate_neighbors(board)
        random.shuffle(candidates)
        improved = False
        for neighbor in candidates:
            neighbor_h = count_conflicts(neighbor)
            evaluations += 1
            if neighbor_h < current_h:
                board = neighbor
                current_h = neighbor_h
                iterations += 1
                improved = True
                if verbose:
                    print("step", iterations, ":", board, " h =", current_h)
                break
        if not improved:
            break
    return board, current_h, iterations, evaluations

if __name__ == "__main__":
    random.seed(0)
    initial = create_random_board()
    print("First-Choice Hill Climbing")
    print("Start board :", initial, " h =", count_conflicts(initial))
    final, h_val, iters, evals = first_choice_climb(initial, verbose=True)
```

```
show_summary("First-Choice Hill Climbing", initial, final, h_val, iters, evals)
```

Output :

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> python .\first_choice_hc.py
First-Choice Hill Climbing
Start board : [6, 6, 0, 4, 7, 6, 4, 7]  h = 9
step 1 : [5, 6, 0, 4, 7, 6, 4, 7]  h = 8
step 2 : [5, 6, 0, 4, 7, 1, 4, 7]  h = 5
step 3 : [5, 6, 0, 5, 7, 1, 4, 7]  h = 4
step 4 : [5, 6, 0, 5, 7, 1, 3, 7]  h = 3
step 5 : [5, 2, 0, 5, 7, 1, 3, 7]  h = 2
step 6 : [5, 2, 0, 5, 7, 1, 3, 6]  h = 1
step 7 : [4, 2, 0, 5, 7, 1, 3, 6]  h = 0

==== First-Choice Hill Climbing ====
Start board : [6, 6, 0, 4, 7, 6, 4, 7]  h = 9
Final board : [4, 2, 0, 5, 7, 1, 3, 6]  h = 0
Result      : SUCCESS
Iterations  : 7
Neighbour evaluations : 94

. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . . . Q .
Q . . . . .
. . . Q . . .
. . . . . Q
. . . . Q . .

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> █
```

4. stochastic.py — Task 5

```
# TASK 5 : Stochastic Hill Climbing
# Idea: 1. Find ALL neighbours better than current. 2. Pick ONE randomly. 3. Stop if none.
```

```

import random
from nqueens_utils import count_conflicts, generate_neighbors, create_random_board, show_summary

def stochastic_climb(board, verbose=False):
    iterations = 0
    evaluations = 0
    current_h = count_conflicts(board)
    while current_h > 0:
        improving = []
        for neighbor in generate_neighbors(board):
            neighbor_h = count_conflicts(neighbor)
            evaluations += 1
            if neighbor_h < current_h:
                improving.append(neighbor)
        if not improving:
            break
        board = random.choice(improving)
        current_h = count_conflicts(board)
        iterations += 1
        if verbose:
            print("step", iterations, ":", board, " h =", current_h,
                  " (picked from", len(improving), "better neighbours)")
    return board, current_h, iterations, evaluations

if __name__ == "__main__":
    random.seed(8)
    initial = create_random_board()
    print("Stochastic Hill Climbing")
    print("Start board :", initial, " h =", count_conflicts(initial))
    final, h_val, iters, evals = stochastic_climb(initial, verbose=True)
    show_summary("Stochastic Hill Climbing", initial, final, h_val, iters, evals)

```

Output :

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> python .\stochastic_hc.py
Stochastic Hill Climbing
Start board : [3, 5, 6, 2, 3, 0, 1, 2]  h = 9
step 1 : [3, 5, 6, 2, 5, 0, 1, 2]  h = 8  (picked from 23 better neighbours)
step 2 : [3, 5, 6, 2, 5, 0, 1, 0]  h = 6  (picked from 23 better neighbours)
step 3 : [3, 5, 6, 2, 5, 1, 1, 0]  h = 5  (picked from 15 better neighbours)
step 4 : [3, 5, 6, 2, 5, 1, 4, 0]  h = 3  (picked from 11 better neighbours)
step 5 : [3, 5, 6, 2, 5, 7, 4, 0]  h = 2  (picked from 6 better neighbours)
step 6 : [3, 1, 6, 2, 5, 7, 4, 0]  h = 0  (picked from 1 better neighbours)

==== Stochastic Hill Climbing ====
Start board : [3, 5, 6, 2, 3, 0, 1, 2]  h = 9
Final board : [3, 1, 6, 2, 5, 7, 4, 0]  h = 0
Result      : SUCCESS
Iterations  : 6
Neighbour evaluations : 336

. . . . . Q
. Q . . . . .
. . . Q . . . .
Q . . . . .
. . . . . Q .
. . . . Q . .
. . . Q . . .
. . Q . . . .
. . . . Q . .

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> █
```

5. random_restart.py — Task 6

```
# TASK 6 : Random Restart Hill Climbing
# Idea: 1. Run Steepest Climb. 2. If stuck ( $h > 0$ ), restart from random board. 3. Continue up to 100
restarts.
```

```

import random
from nqueens_utils import count_conflicts, create_random_board, show_summary
from steepest_hill import steepest_climb

RESTART_LIMIT = 100

def restart_climb(board, verbose=False):
    cumulative_iters = 0
    cumulative_evals = 0
    restart_count = 0
    while True:
        solution, h_val, iters, evals = steepest_climb(board)
        cumulative_iters += iters
        cumulative_evals += evals
        if verbose:
            print("climb", restart_count, ": ended at", solution, " h =", h_val)
        if h_val == 0:
            break
        if restart_count >= RESTART_LIMIT:
            break
        restart_count += 1
        board = create_random_board()
    return solution, h_val, cumulative_iters, cumulative_evals, restart_count

if __name__ == "__main__":
    random.seed(9)
    initial = create_random_board()
    print("Random Restart Hill Climbing")
    print("Start board :", initial, " h =", count_conflicts(initial))
    final, h_val, iters, evals, restarts = restart_climb(initial, verbose=True)
    show_summary("Random Restart Hill Climbing", initial, final, h_val, iters, evals)
    print("Restarts used :", restarts)

```

Output :

```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION> python .\restart_hill.py
Random Restart Hill Climbing
Start board : [7, 5, 4, 2, 2, 0, 5, 7]  h = 8
climb 0 : ended at [7, 1, 4, 6, 2, 0, 5, 3]  h = 1
climb 1 : ended at [1, 5, 0, 0, 3, 7, 6, 2]  h = 2
climb 2 : ended at [5, 3, 0, 4, 7, 1, 6, 2]  h = 0

===== Random Restart Hill Climbing =====
Start board : [7, 5, 4, 2, 2, 0, 5, 7]  h = 8
Final board : [5, 3, 0, 4, 7, 1, 6, 2]  h = 0
Result      : SUCCESS
Iterations  : 9
Neighbour evaluations : 616
Restarts used : 2
PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMITION>

```

6. run_experiments.py — Experiments

```
# EXPERIMENTS : run each algorithm 20 times and print the observation table.
# All four algorithms start from the SAME 20 random boards so comparison is fair. random.seed(21).
import random
from nqueens_utils import count_conflicts, create_random_board
from steepest_hill import steepest_climb
from first_choice_hc import first_choice_climb
from stochastic_hc import stochastic_climb
from restart_hill import restart_climb

NUM_TRIALS = 20

def benchmark_algorithm(algo_label, algo_fn, boards):
    print()
    print("--- " + algo_label + " : " + str(NUM_TRIALS) + " runs ---")
    print("Run | Start h | Result | Final h | Iters | Evals")
    wins = 0
    sum_iters = 0
    sum_evals = 0
    sum_h = 0

    for idx in range(NUM_TRIALS):
        test_board = boards[idx].copy()
        if algo_label == "Random Restart":
            final, h_val, iters, evals, _ = algo_fn(test_board)
        else:
            final, h_val, iters, evals = algo_fn(test_board)

        outcome = "Success" if h_val == 0 else "Failure"
        if h_val == 0:
            wins += 1
        sum_iters += iters
        sum_evals += evals
        sum_h += h_val

        print("%3d | %7d | %-7s | %7d | %5d | %6d"
              % (idx + 1, count_conflicts(boards[idx]), outcome, h_val, iters, evals))

    return wins, sum_iters / NUM_TRIALS, sum_evals / NUM_TRIALS, sum_h / NUM_TRIALS
```

```

# ---- main execution ----
if __name__ == "__main__":
    random.seed(21)
    boards = [create_random_board() for _ in range(NUM_TRIALS)]
    algorithms = [
        ("Steepest Ascent", steepest_climb),
        ("First-Choice", first_choice_climb),
        ("Stochastic", stochastic_climb),
        ("Random Restart", restart_climb),
    ]

    summary_rows = []
    for label, fn in algorithms:
        row = (label,) + benchmark_algorithm(label, fn, boards)
        summary_rows.append(row)

    print()
    print("===== OBSERVATION TABLE =====")
    print("(seed = 21, " + str(NUM_TRIALS) + " runs per algorithm)")
    print()
    print("Algorithm          | Successes (20) | Avg Iterations | Avg Evaluations | Avg Final h")
    print("-" * 82)
    for label, wins, avg_it, avg_ev, avg_h in summary_rows:
        print("%-16s | %14d | %14.2f | %15.2f | %11.2f"
              % (label, wins, avg_it, avg_ev, avg_h))

```

Outputs :

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> python .\run_experiments.py
--- Steepest Ascent : 20 runs ---
Run | Start h | Result | Final h | ITERS | Evals
  1 |      6 | Failure |      1 |     4 |   280
  2 |      8 | Failure |      1 |     3 |   224
  3 |      8 | Failure |      1 |     3 |   224
  4 |      8 | Failure |      2 |     2 |   168
  5 |     10 | Failure |      1 |     3 |   224
  6 |     11 | Failure |      1 |     4 |   280
  7 |      7 | Failure |      1 |     4 |   280
  8 |      7 | Failure |      1 |     3 |   224
  9 |      7 | Failure |      2 |     3 |   224
 10 |     10 | Success |      0 |     5 |   280
 11 |      5 | Success |      0 |     4 |   224
 12 |      5 | Failure |      1 |     3 |   224
 13 |      8 | Failure |      2 |     3 |   224
 14 |      4 | Success |      0 |     3 |   168
 15 |      7 | Failure |      2 |     2 |   168
 16 |      7 | Failure |      1 |     3 |   224
 17 |      9 | Failure |      2 |     3 |   224
 18 |      8 | Failure |      1 |     3 |   224
 19 |      7 | Failure |      1 |     3 |   224
 20 |      9 | Failure |      1 |     3 |   224

--- First-Choice : 20 runs ---
Run | Start h | Result | Final h | ITERS | Evals
  1 |      6 | Failure |      2 |     3 |    69
  2 |      8 | Failure |      1 |     5 |    80
  3 |      8 | Failure |      1 |     6 |   151
  4 |      8 | Failure |      1 |     6 |   137
  5 |     10 | Success |      0 |     6 |    54
  6 |     11 | Success |      0 |    10 |    95
  7 |      7 | Success |      0 |     5 |    39
  8 |      7 | Failure |      3 |     3 |   102
  9 |      7 | Failure |      1 |     5 |   150
 10 |     10 | Failure |      1 |     9 |   122
 11 |      5 | Failure |      2 |     3 |   108
 12 |      5 | Failure |      2 |     3 |    82
 13 |      8 | Failure |      2 |     6 |    75
 14 |      4 | Success |      0 |     3 |    25
 15 |      7 | Failure |      2 |     4 |    69
 16 |      7 | Success |      0 |     6 |    97
 17 |      9 | Failure |      2 |     5 |    85
 18 |      8 | Failure |      1 |     5 |    96
 19 |      7 | Failure |      1 |     5 |    91
 20 |      9 | Failure |      1 |     6 |    95
```

--- Stochastic : 20 runs ---

Run	Start h	Result	Final h	Iters	Evals
1	6	Failure	1	4	280
2	8	Failure	1	5	336
3	8	Failure	2	5	336
4	8	Failure	2	5	336
5	10	Failure	1	7	448
6	11	Failure	1	7	448
7	7	Failure	1	6	392
8	7	Failure	1	4	280
9	7	Success	0	6	336
10	10	Failure	1	7	448
11	5	Failure	2	3	224
12	5	Failure	1	3	224
13	8	Failure	1	6	392
14	4	Failure	1	2	168
15	7	Failure	2	4	280
16	7	Failure	2	3	224
17	9	Failure	1	6	392
18	8	Success	0	5	280
19	7	Failure	2	4	280
20	9	Failure	2	6	392

--- Random Restart : 20 runs ---

Run	Start h	Result	Final h	Iters	Evals
1	6	Success	0	15	1008
2	8	Success	0	38	2800
3	8	Success	0	11	728
4	8	Success	0	16	1176
5	10	Success	0	17	1120
6	11	Success	0	32	2184
7	7	Success	0	15	1008
8	7	Success	0	45	3192
9	7	Success	0	45	3192
10	10	Success	0	5	280
11	5	Success	0	4	224
12	5	Success	0	11	728
13	8	Success	0	10	672
14	4	Success	0	3	168
15	7	Success	0	9	616
16	7	Success	0	33	2296
17	9	Success	0	11	728
18	8	Success	0	8	504
19	7	Success	0	17	1176
20	9	Success	0	38	2744

===== OBSERVATION TABLE =====

(seed = 21, 20 runs per algorithm)

Algorithm	Successes (20)	Avg Iterations	Avg Evaluations	Avg Final h
Steepest Ascent	3	3.20	226.80	1.10
First-Choice	5	5.20	91.10	1.15
Stochastic	2	4.90	324.80	1.25
Random Restart	20	19.15	1327.20	0.00

PS C:\Users\KHATRI OM KUMAR\Desktop\AI-SUBMISSION> █

Discussion Questions :

1. Which algorithm solved the puzzle most often?

Answer:

Random Restart solved the puzzle the most. It solved 20 out of 20 runs, so **100% success**.

- The other three algorithms solved only about 2 to 5 runs out of 20.
- The reason Random Restart works well is simple: if one starting position gets stuck, it starts again from a new random position. So even if one attempt fails, another attempt can succeed.

2. Which required the fewest neighbour evaluations?

Answer:

First-Choice Hill Climbing required the fewest neighbour evaluations.

- First-Choice → **91.10** evaluations/run
- Steepest Ascent → **226.80** evaluations/run
- Stochastic → **324.80** evaluations/run
- The main reason is that First-Choice stops as soon as it finds one better neighbour.
- Steepest Ascent checks all 56 possible neighbours and then chooses the best one.
- So First-Choice saves a lot of computation.

3. Why does Steepest Ascent get stuck?

Answer:

Steepest Ascent gets stuck because it only looks at the current position and its immediate neighbours.

- Sometimes it reaches a local optimum. This means:
- The current board is not the solution, but none of its neighbouring boards is better.
- So the algorithm stops because it cannot find a better move.
- For example, suppose: Current $h = 1$ and every neighbouring board has $h \geq 1$. The algorithm says, "There is no better move," and terminates, even though the actual solution requires $h = 0$.
- Steepest Ascent does not allow a temporarily worse move to escape.

4. How does Random Restart overcome local optima?

Answer:

Random Restart doesn't try to escape the local optimum directly.

- Instead, it throws away the stuck solution and starts fresh from a completely new random board.
- Start $\rightarrow$ climb $\rightarrow$ local optimum $\rightarrow$ stuck $\rightarrow$ Restart with new random board $\rightarrow$ climb again.
- By repeating restarts, it is mathematically likely to start from a basin of attraction that leads directly to the global optimum ($h = 0$).

5. Why can Stochastic Hill Climbing outperform Steepest Ascent?

Answer:

Steepest Ascent always greedily chooses the single best immediate move, which often drives the search into a steep local optimum trap. Stochastic Hill Climbing randomly selects among all improving neighbours. Even if a move gives a smaller immediate reduction in h , it can lead along a different branch of the state space that avoids traps and reaches $h = 0$.

→ Conclusion :

All four algorithms were tested on the 8-Queens problem. The three standard hill-climbing methods frequently get trapped in local optima, whereas **Random Restart guarantees completeness** given sufficient restarts at the cost of higher cumulative evaluations.